BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai

Work Integrated Learning Programmes Division
M.Tech in Artificial Intelligence & Machine Learning

Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 1

Heart Disease Risk Prediction

An end-to-end MLOps pipeline: experiment tracking, CI/CD, containerised serving, Kubernetes deployment and monitoring

Submitted by	Prashob Narendran
Student ID	2025AE05006
Email	2025ae05006@wilp.bits-pilani.ac.in
Code repository	github.com/prashob-narendran/heart-disease-mlops
Pipeline video	drive.google.com/drive/folders/16mSTZy-wzmE14V36OrJE33gThXayBDOc
Submission date	18 October 2026

Instructor In-Charge	Prof. A. Abdul Rahman (arahman@wilp.bits-pilani.ac.in)
Section faculty	Prof. Pranav Sanjay Barbade (pranavsanjayb@wilp.bits-pilani.ac.in)
Lab faculty	Prof. Priyanka Kumari (priyanka.kumari@wilp.bits-pilani.ac.in)

Contents

1. Introduction and architecture....3
2. Setup, installation and access....3
3. Data acquisition and exploratory analysis....4
4. Feature engineering and model development....5
5. Experiment tracking with MLflow....5
6. Model packaging and reproducibility....6
7. CI/CD pipeline and automated testing....6
8. Model containerisation....7
9. Production deployment on Kubernetes....7
10. Monitoring and logging....8
11. Production-readiness verification....10
12. Challenges, lessons learned and conclusion....11
Appendix A. Repository map and screenshot index....13
Appendix B. Deployment and monitoring evidence....14
Appendix C. Virtual lab evidence....15

Summary

I built a heart-disease risk classifier on the UCI Cleveland dataset and took it all the way from raw data to a monitored service on Kubernetes. Every step is automated and reproducible: the dataset is downloaded and checksummed by a script, four models are tuned with cross-validation and tracked in MLflow, the selected model is packaged as a single scikit-learn pipeline, and a five-stage GitHub Actions pipeline lints, tests, retrains, quality-gates, containerises, deploys to a throw-away Kubernetes cluster and verifies monitoring on every push.

Table 1. *Key results at a glance.*

Area	Result
Model	SVM (RBF), C = 10, γ = 0.01, balanced class weights. CV ROC-AUC **0.901 ± 0.019**, CV recall 0.792; test ROC-AUC **0.954**, test recall **0.893** (3 missed cases out of 28)
Tests	**70** pytest tests (data, model, inference, end-to-end, API, metrics), ~90 % coverage (API 98 %), flake8 clean
CI/CD	5 jobs: lint → test → train + quality gate → Docker build/run/smoke test (publish to GHCR) → Kubernetes (kind) deploy, rollback and monitoring check. ~8 min per run; any failure stops the pipeline
Serving	FastAPI `/predict`, `/predict/batch`, `/health`, `/model-info`, `/metrics`; strict input validation (HTTP 422); non-root, read-only Docker image with HEALTHCHECK
Deployment	Docker Desktop Kubernetes v1.36: 2–5 replicas behind a LoadBalancer, zero-downtime rolling updates, rollback, self-healing in ~8 s, autoscaling 2 → 5 pods under load
Monitoring	Prometheus (pod discovery, 7 unit-tested alert rules incl. prediction drift) + Grafana (16-panel dashboard), structured JSON request logs
Reproducibility	Re-run from a clean clone on the BITS Prayogshala virtual lab (Rocky Linux) with `scripts/lab_verify.sh` on 17 Oct 2026: byte-identical dataset, identical model and metrics, Docker and failure checks passed, all 5 stages PASSED in about 7 minutes

1. Introduction and architecture

Problem. Given 13 routinely collected clinical attributes of a patient (age, sex, chest-pain type, blood pressure, cholesterol, ECG results, exercise test results, fluoroscopy and thallium results), predict whether heart disease is present, and serve that prediction as a reliable, monitored API. In a screening setting a missed disease case is far more costly than a false alarm, so I treat **recall** and **ROC-AUC** as the primary metrics throughout.

Architecture. Figure 1 shows the three layers of the solution. Development happens on my Windows laptop (and is re-verified on the BITS Linux virtual lab). Every push to GitHub triggers the CI/CD pipeline, which retrains and quality-gates the model, builds and tests the container, publishes it to the GitHub Container Registry (GHCR) and proves it deploys on Kubernetes. The production deployment runs on Docker Desktop's Kubernetes, with Prometheus and Grafana for monitoring.

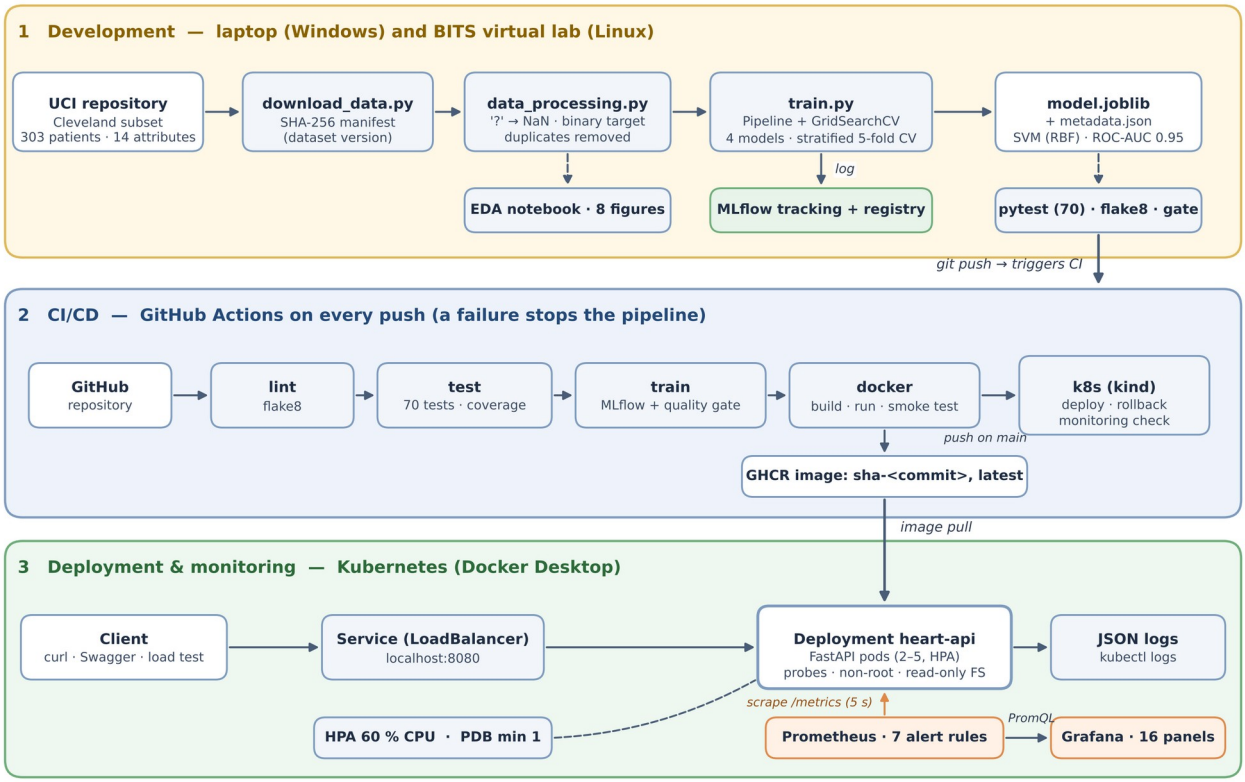


Figure 1. *Solution architecture: development and tracking (1), CI/CD with GitHub Actions (2), Kubernetes deployment and monitoring (3).*

Table 2. *Technology stack (all versions pinned).*

Concern	Tools
Data & modelling	Python 3.11, pandas 2.2, scikit-learn 1.5.2, Jupyter, matplotlib/seaborn
Experiment tracking & registry	MLflow 2.17 (file store, nested runs, model registry with `champion` alias)
Testing & quality	pytest + pytest-cov, flake8, model quality gate (`scripts/check_model.py`)
Serving & packaging	FastAPI 0.115, pydantic 2, uvicorn; joblib pipeline; Docker (python:3.11-slim)
CI/CD	GitHub Actions (5 jobs), GHCR, kind v0.31 for Kubernetes tests in CI
Deployment	Kubernetes 1.36 (Docker Desktop, kubeadm), Kustomize base + overlays, HPA, PDB, metrics-server
Monitoring	prometheus-client, Prometheus 3.5 LTS, Grafana 12.4, structured JSON logs

2. Setup, installation and access

Prerequisites: Git, Python 3.11 and Docker Desktop (with Kubernetes enabled for section 9). Everything else comes from the pinned `requirements.txt`. The commands below are for Windows PowerShell; the Linux equivalents are in the README.

```
git clone https://github.com/prashob-narendran/heart-disease-mlops.git
cd heart-disease-mlops ; py -3.11 -m venv .venv ; .venv\Scripts\activate
```

```
pip install -r requirements.txt
.\scripts\reproduce.ps1          # checksum, clean, EDA, lint, 70 tests, train, gate, predict
mlflow ui                          # experiment tracking: http://127.0.0.1:5000
docker build -t heart-api . ; docker run -d -p 8000:8000 --name heart-api heart-api
python scripts/smoke_test_api.py   # 6 end-to-end checks on http://localhost:8000
.\scripts\k8s_deploy.ps1 -Local    # Kubernetes: build, deploy, rollout, smoke test
kubectl apply -k k8s/monitoring/   # Prometheus :9090 + Grafana :3000
```

Access instructions (local deployment). The API is not hosted publicly; after the steps above it is available at **http://localhost:8080** (Kubernetes Service) or **http://localhost:8000** (single container). Interactive documentation is at `/docs`. A prediction request looks like:

```
curl.exe -X POST http://localhost:8080/predict -H "Content-Type: application/json" ^
         -d "@samples/sample_patient.json"
{"prediction":1, "label":"disease", "disease_probability":0.5518, "confidence":0.5518,
 "risk_level":"moderate", "model_name":"svm_rbf", "model_version":4}
```

Each step was also re-executed on a fresh Linux machine (section 11), and step-by-step re-execution guides for every stage are in the repository.

3. Data acquisition and exploratory analysis

Dataset choice. The assignment names the UCI Heart Disease dataset. I use the **Cleveland subset** (`processed.cleveland.data`, 303 patients, 14 attributes) because it is the only one of the four UCI sites with near-complete data (the Hungarian, Swiss and VA files lack most `ca`/`thal`/`slope` values) and it is the subset used in published work. I deliberately did not use the popular Kaggle `heart.csv` copy: while checking it I found it re-encodes some codes (for example `ca = 4` and `thal = 0`, which do not exist in the UCI documentation), so a model trained on it would accept impossible inputs.

Acquisition. `scripts/download_data.py` fetches the raw file from the UCI archive (falling back to the official `ucimlrepo` client), validates its structure (303 rows × 14 fields) and writes a manifest with its **SHA-256 checksum**. The checksum is my lightweight dataset version: it is verified before every pipeline run, logged with every MLflow run, and stored in the model metadata. On Windows, Git initially converted line endings and changed the checksum, which I fixed with a `.gitattributes` rule that keeps `data/**` byte-exact.

Table 3. *Cleaning rules (`src/data_processing.py`) and the reason for each.*

Rule	Reason
Assign the 14 documented column names; `?` → NaN	Raw file has no header; missing values are marked `?`
No imputation at this stage	Imputation is fitted inside the model pipeline on training folds only (no leakage), and lets the API accept an unknown `ca`/`thal`
Categorical codes outside the documented sets → NaN	Guards against corrupt or re-encoded inputs
`num` (0–4) → binary `target` (1 if > 0)	The task is presence/absence of disease
Remove exact duplicates	Avoid leakage between CV folds (none found in Cleveland)

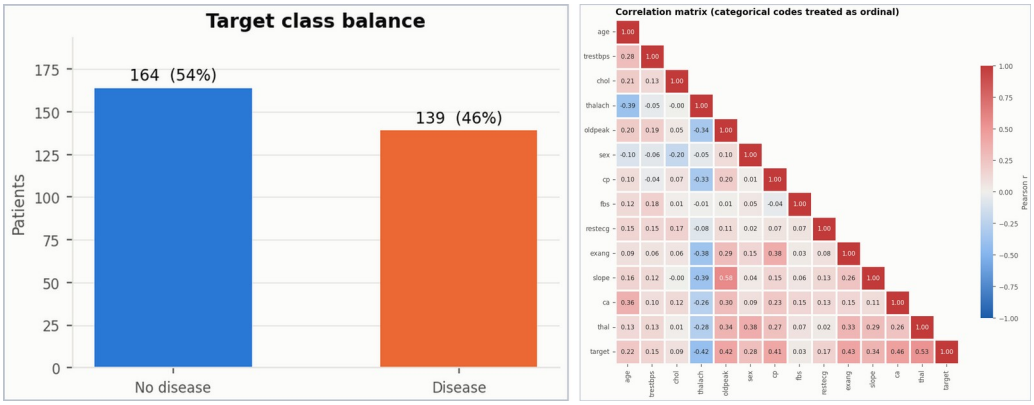


Figure 2. *Class balance (164 vs 139, about 54 % / 46 %) and correlation matrix of all attributes with the target.*

Findings that shaped the model (full analysis in `notebooks/01_eda.ipynb`, 8 figures in `reports/figures/`):

- Only `ca` (4 rows) and `thal` (2 rows) have gaps (~2 %), so I keep all patients and impute inside the pipeline.
- Classes are roughly balanced, so no resampling is needed, but I use **stratified** 5-fold CV and select on ROC-AUC and recall rather than accuracy.
- The strongest associations with disease are `thal`, `ca`, `exang`, `oldpeak`, `cp` and (negatively) `thalach`, each with |r| ≈ 0.4–0.5; no single feature dominates and the largest feature–feature correlation is only 0.58 (`oldpeak`–`slope`), so all 13 features are kept.
- Numeric features live on very different scales (oldpeak 0–6 vs cholesterol 126–564), so they are **standardised**; the 8 categorical codes are **nominal** (e.g. `thal` = 3/6/7) and are **one-hot encoded**. Extreme values (cholesterol 564, oldpeak 6.2) are clinically plausible and are kept.

4. Feature engineering and model development

One pipeline object. Preprocessing and model live in a single scikit-learn `Pipeline`: numeric features → median imputation → `StandardScaler`; categorical features → most-frequent imputation → `OneHotEncoder(handle_unknown="ignore")` (needed because some levels, such as `restecg = 1`, have only 4 patients and may be absent from a fold); then the estimator. Because preprocessing travels with the model, training, CLI inference, the API and CI can never disagree on how a patient is transformed.

Training protocol. A stratified 80/20 split (seed 42) holds out 61 patients as a test set that is used for **reporting only**. Each of four candidate models is tuned with `GridSearchCV` (stratified 5-fold CV, refit on ROC-AUC, also scoring accuracy, precision, recall and F1):

Table 4. *Model comparison (5-fold CV on the training set; held-out test set). Selected model highlighted.*

Model (grid size)	CV ROC-AUC	CV recall	CV F1	Test ROC-AUC	Test recall	Test accuracy
Logistic Regression, L1/L2 (24)	**0.907** ± 0.011	0.783	0.812	0.959	0.893	0.869
Random Forest (36)	0.902 ± 0.033	0.774	0.787	0.948	0.893	0.852
Gradient Boosting (16)	0.885 ± 0.030	0.756	0.792	0.960	0.929	0.902
SVM, RBF kernel (30)	0.901 ± 0.019	**0.792**	**0.818**	0.954	0.893	0.852

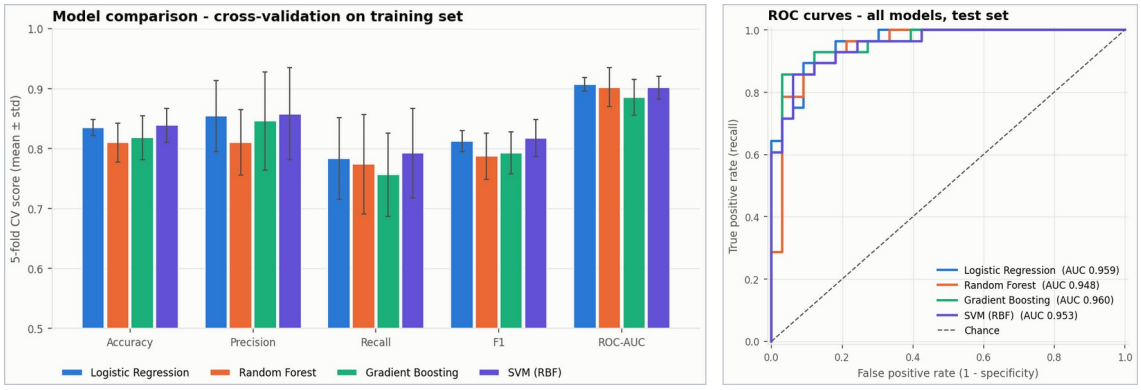


Figure 3. *Cross-validated metrics of the four models (left) and ROC curves on the held-out test set (right).*

Selection rule and why SVM. I fixed the rule before looking at test results: choose the highest CV ROC-AUC, but treat models within 0.01 as tied and break the tie by **higher CV recall**, because a missed disease case costs more than a false alarm. Logistic regression (0.907), random forest (0.902) and SVM (0.901) are statistically indistinguishable on ROC-AUC, so recall decides and the **SVM (C = 10, γ = 0.01, balanced class weights)** wins with the highest recall (0.792) and F1 (0.818). It is also more stable across folds than the random forest (± 0.019 vs ± 0.033). Gradient boosting has the best test scores, but choosing it on that basis would be selecting on the test set, which I avoid. On the test set the SVM misses 3 of 28 disease cases (specificity 0.818).

5. Experiment tracking with MLflow

Every `python -m src.train` creates a **parent run** in the experiment `heart-disease-classification` with one **nested run per model**. Each run logs: parameters (model type, best hyper-parameters, folds, seed, grid size); metrics (CV mean and standard deviation for five metrics, test metrics, specificity, false negatives/positives, fit time); artifacts (confusion matrix, ROC curve, permutation importance as PNG and CSV, full CV results); the fitted pipeline with an input signature and example; and tags (dataset SHA-256, git commit, Python and scikit-learn versions). The selected model is

registered as **`heart-disease-classifier`** and given the alias **`champion`**, which the API can load by alias rather than by version number.

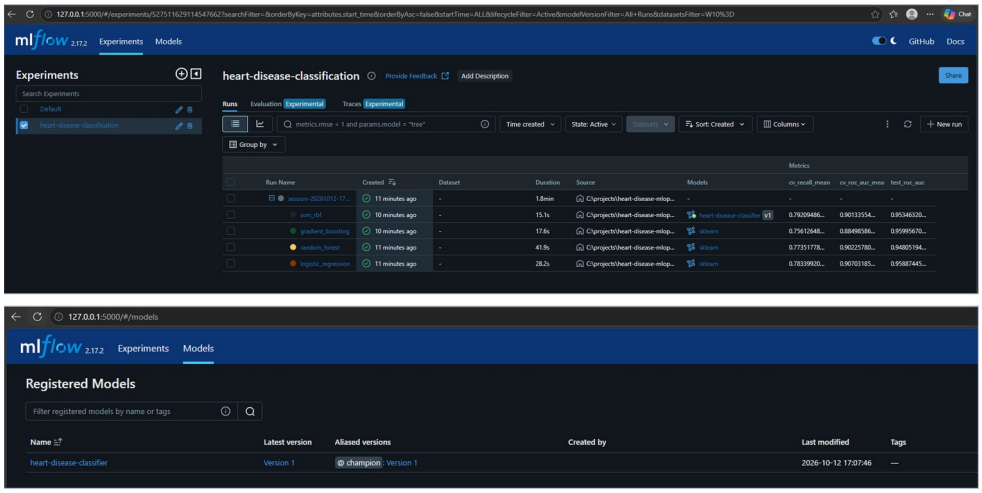


Figure 4. *MLflow: one session with nested runs for the four models (top) and the model registry with the `champion` alias (bottom).*

I also wrapped training as an **MLflow Project** (`MLproject` + `python_env.yaml`), so `mlflow run .` reproduces it in its own environment. During this work a quick CI-style run once overwrote my champion model; I fixed this by sending `--quick` runs to a separate experiment and output folder that never touch the registry.

6. Model packaging and reproducibility

- **Model format:** the whole pipeline is saved as `models/model.joblib` plus `models/metadata.json` (model name, registry version, best parameters, all metrics, decision threshold, dataset SHA-256, git commit, library versions). The same model is also stored in MLflow format with its signature.
- **Environment:** `requirements.txt` pins every package; the slimmer `requirements-api.txt` for the container uses identical versions of numpy, pandas and scikit-learn, because a pickled pipeline must be loaded with the library versions it was trained with.
- **One-command reproduction:** `scripts/reproduce.ps1` / `.sh` runs checksum → clean → EDA → lint → tests → train → quality gate → sample prediction and stops at the first failure.
- **Quality gate:** `scripts/check_model.py` fails the build if CV ROC-AUC < 0.85, CV recall < 0.70 or test ROC-AUC < 0.80, so a degraded model can never be packaged or deployed.

7. CI/CD pipeline and automated testing

Tests. 70 pytest tests cover data cleaning (no network needed), the pipeline and model selection on synthetic data, inference and risk bands, an end-to-end run of `train.py` with MLflow, 28 API tests with FastAPI's `TestClient` (valid input, every validation error → 422, batches, missing values, a 503 when no model is loaded) and the Prometheus metrics. Coverage is about 90 %.

Pipeline (`.github/workflows/ci.yml`, runs on every push and pull request). Each job runs only if the previous one passed:

Table 5. *GitHub Actions jobs, what they prove and what they keep.*

Job	What it does	Artifacts / outputs
1 Lint	flake8 on `src`, `scripts`, `tests`, `api`	annotated errors in the run
2 Test	70 tests with coverage	JUnit XML, coverage XML + HTML
3 Train	verify dataset checksum → clean → train all models with MLflow → quality gate → sample predictions	model, metadata, comparison table, figures, `mlruns/`; results table in the job summary
4 Docker	build the image with the model trained in job 3 → run → smoke test 6 checks → wait for HEALTHCHECK → on `main`, push `sha-<commit>` and `latest` to GHCR	image size/health summary; published image
5 Kubernetes	promtool checks → kind cluster → deploy → smoke test via Service → rolling update + rollback → Prometheus + Grafana → monitoring check	deployment and monitoring summary

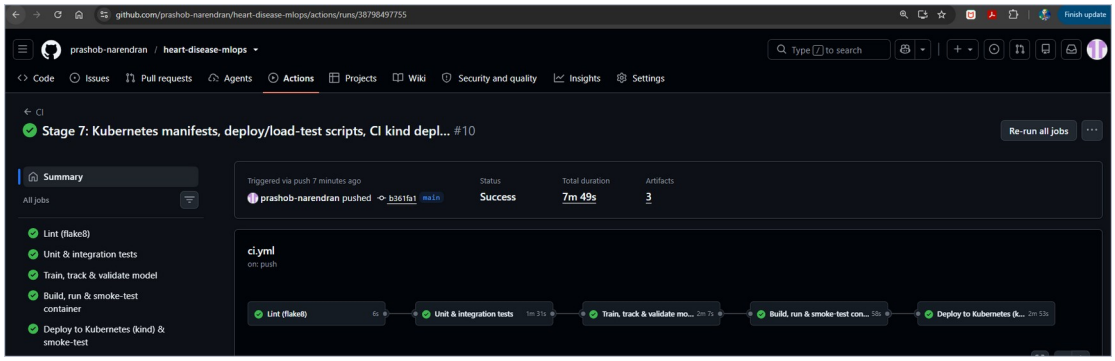


Figure 5. *A successful run: all five jobs green (lint, tests, training, container, Kubernetes) in 7 min 49 s.*

The pipeline fails on code or test errors, with clear logs. I proved this deliberately with two demonstration commits: an unused import (run #4) and a test with a wrong expected value (run #6). In both cases the failing job turned red, the log pointed to the exact file, line and assertion, and all later jobs were skipped, so no model or image was produced. Both commits were then reverted.

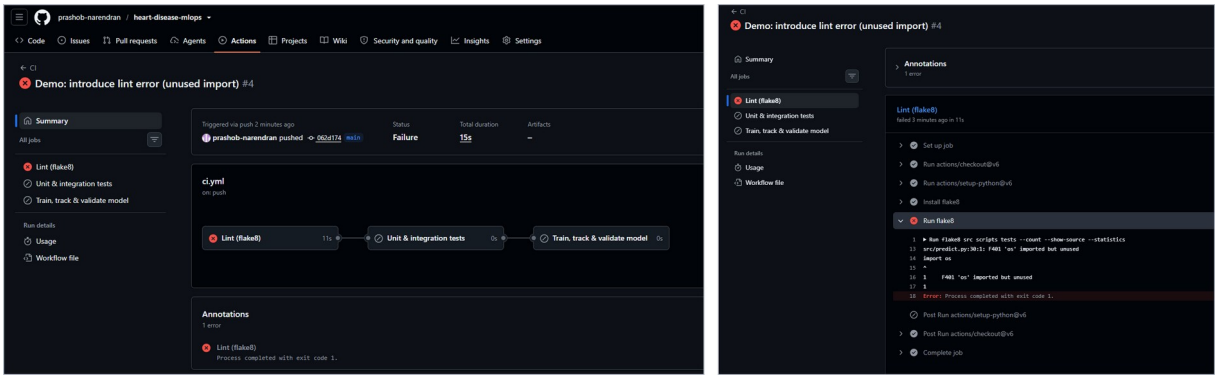


Figure 6. *Run #4: flake8 reports `F401 'os' imported but unused` at `src/predict.py:30`, the job fails and the remaining jobs never start.*

8. Model containerisation

API (`api/main.py`, FastAPI). `POST /predict` takes one patient as JSON and returns the prediction, label, disease probability, **confidence** and a low/moderate/high risk band; `POST /predict/batch` scores up to 100 patients; `GET /health` returns 200 only when the model is loaded (503 otherwise, so orchestrators can react); `GET /model-info` reports the served model and its metrics; `GET /docs` is the interactive Swagger UI. Pydantic models reject impossible inputs with HTTP 422 and the offending field (e.g. `sex = 5`, `ca = 4`, unknown fields), while `ca`/`thal` may be `null` and are imputed. Every response carries `X-Request-ID`, `X-Process-Time-ms` and `X-Served-By` headers.

Image (`Dockerfile`): `python:3.11-slim`, runtime-only dependencies, only code and model copied in (`.dockerignore`), a dedicated non-root user, a `HEALTHCHECK` on `/health`, and uvicorn started with `exec` so it receives shutdown signals as PID 1. It runs unchanged with a read-only root filesystem and all Linux capabilities dropped.

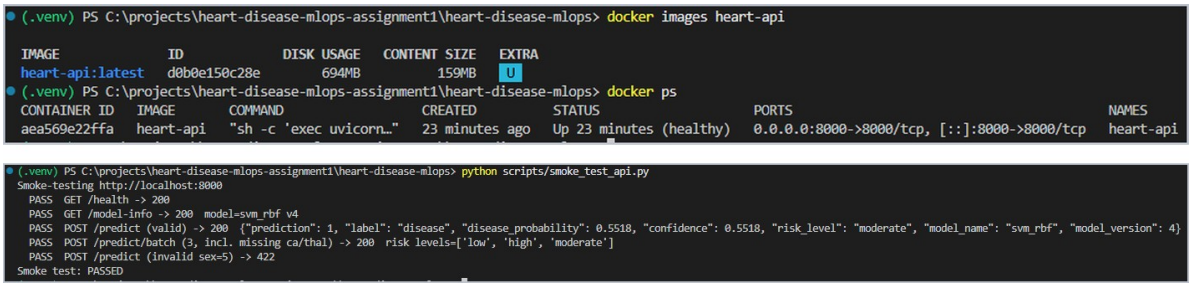


Figure 7. *The container running locally reports (healthy); the smoke test passes every endpoint check, including the 422 for an invalid patient.*

9. Production deployment on Kubernetes

I deployed to **Docker Desktop's Kubernetes** (kubeadm, v1.36), which is the closest local equivalent to a managed cluster, and used plain manifests combined with **Kustomize**: a `base` plus two overlays, one using the CI-published GHCR image and one using a locally built image (also used by CI's kind cluster).

Table 6. *Kubernetes resources (`k8s/base/`).*

Resource	Key settings and why
Namespace	`heart-disease`, Pod Security **restricted** enforced: the API server rejects any pod that runs as root or keeps privileges
Deployment	2 replicas; rolling update with `maxUnavailable 0`, `maxSurge 1` (zero downtime); startup, readiness and liveness probes on `/health`; requests 100m CPU / 256 Mi, limits 500m / 512 Mi; non-root, read-only root FS, all capabilities dropped; `preStop` delay for graceful shutdown
Service	type **LoadBalancer**, port 8080 → container 8000 (http://localhost:8080 on Docker Desktop)
HPA / PDB	2–5 replicas at 60 % average CPU; at least one pod always available during maintenance
ConfigMap	runtime settings (log level, workers) outside the image

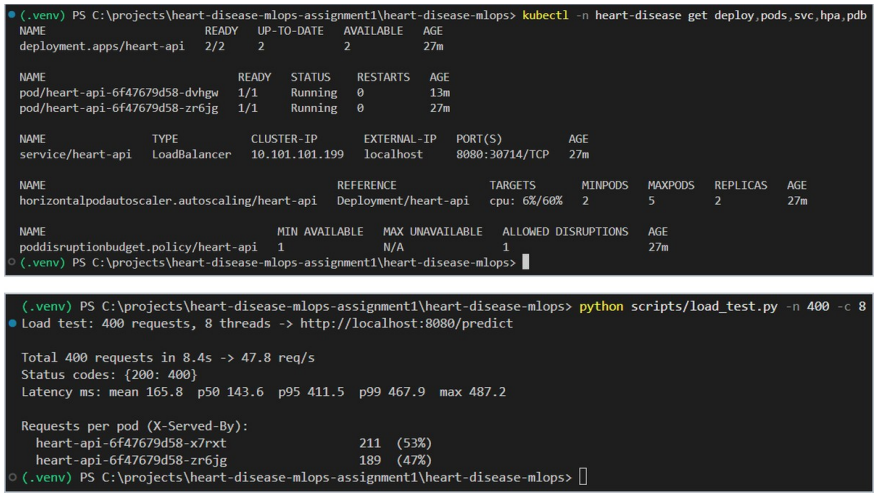


Figure 8. *Deployment 2/2, both pods Running, LoadBalancer on localhost:8080, HPA and PDB (top); 400 requests split 53 % / 47 % across the two pods (bottom).*

Verification. The smoke test passes through the Service; deleting a pod brings a replacement to `Running 1/1` in about 8 s while the other pod keeps serving (self-healing); `kubectl rollout restart` and `rollout undo` update and roll back with zero downtime; and the CI-published image `ghcr.io/prashob-narendran/heart-disease-api:sha-b361fa1` deploys and passes the same smoke test. Under a 3-minute load test the autoscaler reacted to CPU at 452 % of the request by scaling from 2 to 5 pods; all 13,672 requests succeeded and were spread over the five pods, and it scaled back to 2 after the load stopped (Figure 9).

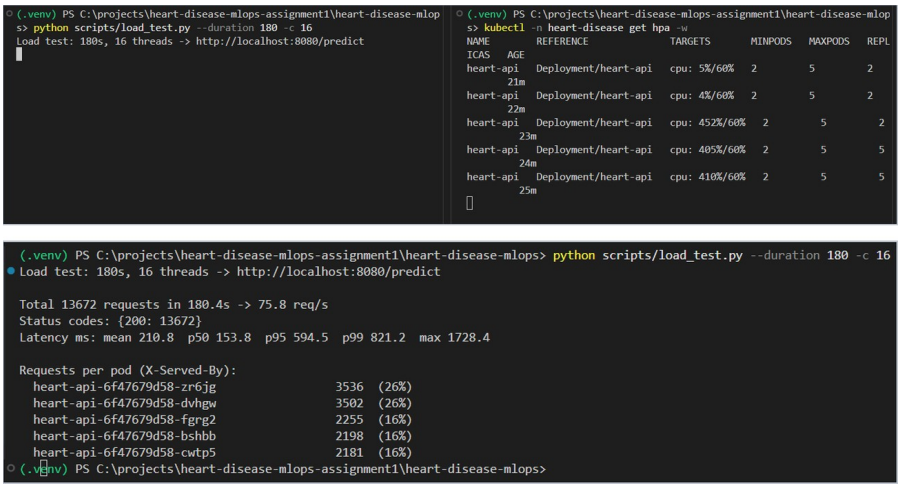


Figure 9. *Autoscaling under load: CPU 452 % → replicas 2 → 5 (top); all 13,672 requests returned 200, spread over five pods (bottom).*

10. Monitoring and logging

Metrics. The API exposes Prometheus metrics at `/metrics`: request count by endpoint and status, a latency histogram, requests in flight, predictions by label and risk level, the distribution of predicted probabilities, batch sizes, and which model is loaded. Endpoint labels use route templates, so a client cannot create unbounded time series.

Prometheus (v3.5) finds every heart-api pod through the Kubernetes API (pods opt in with `prometheus.io/*` annotations; RBAC only allows listing pods in one namespace) and scrapes each Ready pod every 5 s, so pods added by the

autoscaler appear automatically. **Grafana** (12.4) is provisioned from files with the data source and a 16-panel dashboard covering service health, traffic and latency, per-pod load/CPU/memory, model behaviour and active alerts.

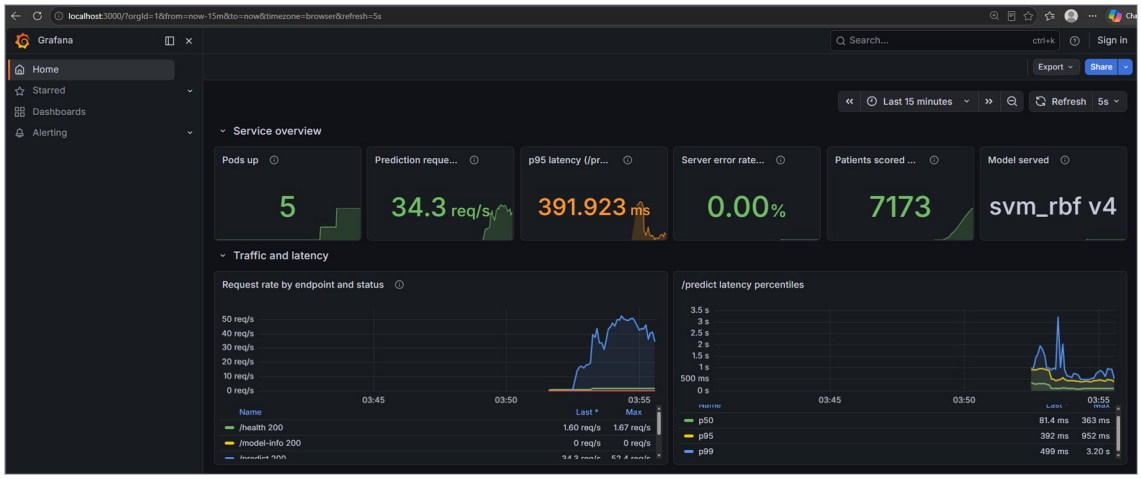


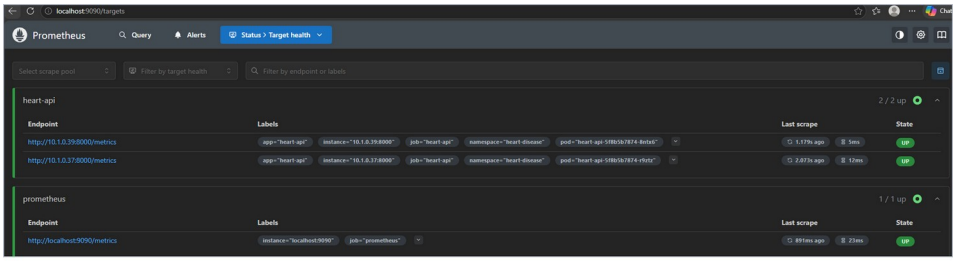
Figure 10. *Grafana dashboard during a load test: 5 pods, 34 prediction requests/s, p95 latency 392 ms, 0 % server errors, 7,173 patients scored, model svm_rbf v4.*

Alerts. Seven alert rules cover availability (no targets, pod down, model not loaded), performance (5xx ratio > 5 %, p95 latency > 500 ms) and the **model** (more than 20 % invalid inputs, and **prediction drift** when more than 80 % of recent patients are predicted as diseased against a training prevalence of ~46 %). Ground truth arrives late in healthcare, so a shift in the prediction mix is a cheap early warning. The rules are unit-tested with `promtool test rules` in CI. I triggered both model alerts deliberately with `load_test.py --drift --invalid 0.3`; they went from pending to **firing** within about 3 minutes (Figure 11).



Figure 11. *Simulated drift and bad input: the disease share jumps to 100 %, 422 responses rise, and both alerts fire.*

Logging. Every request and prediction is written to stdout as one JSON object (timestamp, request id, method, path, status, latency, pod name; prediction, probability and risk), so it can be traced end to end with `kubectl logs` or shipped to any log platform. High-frequency probe and scrape calls are logged only at DEBUG level so they do not drown real traffic.




```
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease logs deployment/heart-api --tail=200 |
>>   Where-Object { $_ -like '{*' } | ForEach-Object { $_ | ConvertFrom-Json } |
>>   Where-Object event -eq 'request' |
>>   Select-Object -Last 10 instance, method, path, status, latency_ms, request_id | Format-Table
Found 2 pods, using pod/heart-api-5f8b5b7874-r9ztz

instance                  method path     status latency_ms request_id
--------                  ------ ----     ------ ---------- ----------
heart-api-5f8b5b7874-r9ztz POST   /predict    422        0.8 2318f9fa582b
heart-api-5f8b5b7874-r9ztz POST   /predict    200      14.17 614ec0e0ec06
heart-api-5f8b5b7874-r9ztz POST   /predict    200      12.02 5288c954f4cb
heart-api-5f8b5b7874-r9ztz POST   /predict    200      12.76 36201307df1b
heart-api-5f8b5b7874-r9ztz POST   /predict    200      11.74 093c4bb2c19c
heart-api-5f8b5b7874-r9ztz POST   /predict    422       0.73 4d102eaa0248
heart-api-5f8b5b7874-r9ztz POST   /predict    200      13.51 600d66f72b51
heart-api-5f8b5b7874-r9ztz POST   /predict    200      11.95 ca87547bacb0
heart-api-5f8b5b7874-r9ztz POST   /predict    200      17.21 cd9d4d6dba98
heart-api-5f8b5b7874-r9ztz POST   /predict    422       0.95 66b2a0118eea
```

Figure 12. *Prometheus scraping both pods (top) and the request log as a table: successful predictions (~12 ms) and rejected invalid inputs (422, < 1 ms).*

11. Production-readiness verification

The assignment requires three properties. Beyond the evidence above, I re-ran the project from scratch on a different machine, the **BITS Prayogshala virtual lab** (Rocky Linux, 4 cores, 15 GiB), using `scripts/lab_verify.sh`, which prints the machine identity at the top of every stage and ends each stage with a PASSED/FAILED banner and a timestamped log.

Table 7. *Production-readiness requirements and evidence.*

Requirement	Evidence
All scripts execute from a clean setup using the requirements file	Fresh clone on the lab VM → new Python 3.11 environment → `pip install -r requirements.txt` → `pip check` → full pipeline and 70 tests pass; the same SVM is selected with identical metrics (CV ROC-AUC 0.901, test 0.954). CI does the same from scratch on every push.
Model serves correctly in an isolated environment (Docker build/test proof)	Image built on the lab VM with the model trained there, run read-only with no capabilities, reports healthy, passes 6/6 smoke checks and returns HTTP 422 for invalid input. CI job 4 repeats this on every push.
Pipeline fails on code or test errors and gives clear logs	CI runs #4 (lint error) and #6 (failing test) stop with exact file/line/assertion and skip all later jobs. On the lab VM, a lint error, a wrong test, a tampered dataset (`Checksum mismatch!`), the full pipeline on tampered data and a too-strict quality gate each stop with a non-zero exit code.

Lab run (17 October 2026, 20:55–21:02 IST). On a fresh clone of commit `8920b66` the fresh UCI download had exactly the same SHA-256 as my versioned dataset, all 70 tests passed (90 % coverage), and training selected the same SVM with the same scores as on my laptop and in CI. The Docker image built on the lab VM served predictions as a non-root `appuser` and passed every smoke check, and all five deliberate failures were caught (Figures 13 and 14; more lab screenshots are in Appendix C).

```
############################################################
 Heart Disease MLOps - lab verification : stage 'summary'
 Date (local)  : 2026-10-17 21:02:29 IST
 User @ host   : s2025ae05006 @ ip-10-3-23-221.ap-south-1.compute.internal
 OS / kernel   : Rocky Linux 9.5 (Blue Onyx) / 5.14.0-503.14.1.el9_5.x86_64 (x86_64)
 CPU / RAM     : 4 cores / 15Gi
 Repository    : https://github.com/prashob-narendran/heart-disease-mlops.git
 Commit        : 8920b66 Add author and submission details to README (2026-10-16)
 Log file      : /home/s2025ae05006/workspace/heart-disease-mlops/lab_proof/20261017_210229_summary.log
############################################################

=== Stage results on this machine ===
  finished            | stage      | result  | log
  2026-10-17 20:55:39 | info       | PASSED  | 20261017_205532_info.log
  2026-10-17 20:57:33 | setup      | PASSED  | 20261017_205539_setup.log
  2026-10-17 21:01:17 | pipeline   | PASSED  | 20261017_205733_pipeline.log
  2026-10-17 21:02:24 | docker     | PASSED  | 20261017_210117_docker.log
  2026-10-17 21:02:28 | failproof  | PASSED  | 20261017_210224_failproof.log

  logs: /home/s2025ae05006/workspace/heart-disease-mlops/lab_proof

################  STAGE 'summary' PASSED  ################
  finished 2026-10-17 21:02:29 IST on ip-10-3-23-221.ap-south-1.compute.internal
(base) s2025ae05006@ip-10-3-23-221:~/workspace/heart-disease-mlops$
```

Figure 13. *Virtual lab: machine identity (user s2025ae05006, host ip-10-3-23-221, Rocky Linux 9.5, commit 8920b66) and all five verification stages PASSED.*

```
  PASS  conda env (python 3.11, conda-forge)
  interpreter: Python 3.11.17 at /home/s2025ae05006/envs/heart-mlops/bin/python

=== 2/4 Install the pinned dependencies from requirements.txt ===
  PASS  pip install -r requirements.txt

=== 3/4 Dependency consistency ===
No broken requirements found.
  PASS  pip check (no broken requirements)

=== 4/4 Key package versions (identical to CI and the Docker image) ===
  python              3.11.17
  numpy               2.1.3
  pandas              2.2.3
  scikit-learn        1.5.2
  mlflow              2.17.2
  fastapi             0.115.6
  pydantic            2.10.3
  uvicorn             0.32.1
  prometheus-client   0.21.1
  pytest              8.3.3

################  STAGE 'setup' PASSED  ################
  finished 2026-10-17 20:57:33 IST on ip-10-3-23-221.ap-south-1.compute.internal
```

```
=== 7/9 Train all models with MLflow tracking (takes a few minutes) ===
INFO | Train 242 rows, test 61 rows, positives 45.9% / 45.9%
INFO | MLflow tracking URI: file:///home/s2025ae05006/workspace/heart-disease-mlops/mlruns
INFO | logistic_regression  cv_auc=0.907±0.011 cv_recall=0.783 test_auc=0.959 test_recall=0.893 (24 combos, 7.4s)
INFO | random_forest        cv_auc=0.902±0.033 cv_recall=0.774 test_auc=0.948 test_recall=0.893 (36 combos, 44.8s)
INFO | gradient_boosting    cv_auc=0.885±0.030 cv_recall=0.756 test_auc=0.960 test_recall=0.929 (16 combos, 10.3s)
INFO | svm_rbf             cv_auc=0.901±0.019 cv_recall=0.792 test_auc=0.953 test_recall=0.893 (30 combos, 6.5s)
Successfully registered model 'heart-disease-classifier'.
Created version '1' of model 'heart-disease-classifier'.
INFO | Model comparison:
                     cv_roc_auc_mean  cv_roc_auc_std  cv_recall_mean  cv_f1_mean  test_roc_auc  test_recall  test_accuracy  selected
logistic_regression            0.907           0.011           0.783       0.812         0.959        0.893          0.869     False
random_forest                  0.902           0.033           0.774       0.787         0.948        0.893          0.852     False
gradient_boosting              0.885           0.030           0.756       0.792         0.960        0.929          0.902     False
svm_rbf                        0.901           0.019           0.792       0.818         0.953        0.893          0.852      True
INFO | Selected: svm_rbf (registered version 1)
INFO | Saved /home/s2025ae05006/workspace/heart-disease-mlops/models/model.joblib
  PASS  src.train

=== 8/9 Model quality gate ===
Model: svm_rbf  (trained 2026-10-17T15:31:12+00:00)
  PASS  cv_roc_auc_mean  0.901  (min 0.85)
  PASS  cv_recall_mean   0.792  (min 0.70)
  PASS  test_roc_auc     0.954  (min 0.80)
Quality gate: PASSED
  PASS  check_model.py
```

Figure 14. *Virtual lab: fresh Python 3.11 environment from requirements.txt with `pip check` clean (top), and retraining with MLflow: the same four models, SVM selected, quality gate passed (bottom).*

12. Challenges, lessons learned and conclusion

Table 8. *Problems I met and how I solved them.*

Problem	Fix
Kaggle copy of the dataset contains impossible codes (`ca = 4`, `thal = 0`)	Used the primary UCI source via a script with checksum; API validation rejects those codes
A quick CI-style training run overwrote my champion model	Quick runs go to a separate MLflow experiment and output folder; registry untouched

Problem	Fix
Windows line endings changed the dataset checksum after a Git checkout	`.gitattributes`: `data/** -text` keeps the data byte-exact on every OS
Matplotlib's Tk backend crashed tests on Windows	Non-interactive `Agg` backend for scripts and tests
New GHCR packages are private, so Kubernetes could not pull the image	Package made public; a `local` Kustomize overlay deploys a locally built image without any registry
Prometheus briefly reported starting/terminating pods as down	Scrape only Ready pods (the same pods the Service sends traffic to); alerts use `for:` windows

Conclusion. The project delivers a reproducible, tested and observable ML service: a defensible model choice driven by recall and cross-validation, full experiment lineage in MLflow, a pipeline that refuses to ship broken code, data or models, a hardened container, a self-healing and autoscaling Kubernetes deployment, and monitoring that watches both the service and the model's behaviour. The main lessons for me were that reproducibility depends on small details (line endings, library versions, isolated test runs), and that monitoring the model's outputs is as important as monitoring the service. Natural next steps are an Alertmanager route for the alerts, a persistent volume for Prometheus, ingesting delayed ground-truth labels to monitor real accuracy, and canary releases driven by the registry alias.

Code repository: https://github.com/prashob-narendran/heart-disease-mlops · **Pipeline video (Google Drive):** drive.google.com/drive/folders/16mSTZy-wzmE14V36OrJE33gThXayBDOc

Appendix A. Repository map and screenshot index

The report shows the key evidence only. Every screenshot from every stage is in the repository's `screenshots/` folder, indexed below.

Table 9. *Where each deliverable lives in the repository.*

Deliverable	Location
Code (data, features, models, training, inference)	`src/`
Dockerfile, requirements	`Dockerfile`, `.dockerignore`, `requirements.txt`, `requirements-api.txt`, `python_env.yaml`, `MLproject`
Dataset, download script, cleaned data	`scripts/download_data.py`, `data/raw/` (+ SHA-256 manifest), `data/processed/heart_clean.csv`
Notebooks and scripts (EDA, training, inference)	`notebooks/01_eda.ipynb`, `src/train.py`, `src/predict.py`, `scripts/`
Unit and integration tests	`tests/` (70 tests)
CI/CD workflow	`.github/workflows/ci.yml`
Deployment manifests	`k8s/base/`, `k8s/kustomization.yaml`, `k8s/local/`, `k8s/addons/metrics-server/`
Monitoring	`api/metrics.py`, `k8s/monitoring/` (Prometheus config, alert rules + tests, Grafana dashboard)
Screenshots	`screenshots/` (s1–s8 per stage, lab_* for the virtual lab)
Model and reports	`models/model.joblib`, `models/metadata.json`, `reports/model_comparison.csv`, `reports/figures/`

Table 10. *Screenshot index (`screenshots/` folder).*

Prefix	Stage	Highlights
s1_*	Data and EDA	download with checksum, cleaning, tests, notebook findings and charts
s2_*	Models and MLflow	training output, runs, comparison, run details, artifacts, registry
s4_*	Packaging	coverage, quality gate, CLI prediction, one-command reproduction
s5_*	CI/CD	run history, failed lint/test runs with logs, passed run, summary, artifacts, badge
s6_*	Container	Swagger, Docker build, healthy container, smoke test, curl 200/422, logs, CI docker job, GHCR
s7_*	Kubernetes	nodes, deploy, resources, smoke test, load balancing, self-healing, HPA, rollout/undo, CI kind job, GHCR deploy
s8_*	Monitoring	70 tests, /metrics, Prometheus targets/query/alerts, Grafana, alerts firing, logs, CI monitoring
lab_*	Virtual lab	clean-room verification on BITS Prayogshala

Appendix B. Deployment and monitoring evidence

```
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease get pods
NAME                              READY   STATUS    RESTARTS   AGE
heart-api-6f47679d58-x7rxt   1/1     Running   0          13m
heart-api-6f47679d58-zr6jg   1/1     Running   0          13m
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease delete pod heart-api-6f47679d58-x7rxt
pod "heart-api-6f47679d58-x7rxt" deleted from heart-disease namespace
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops>
```

```
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease get pods -w
NAME                              READY   STATUS              RESTARTS   AGE
heart-api-6f47679d58-x7rxt   1/1     Running             0          11m
heart-api-6f47679d58-zr6jg   1/1     Running             0          10m
heart-api-6f47679d58-x7rxt   1/1     Terminating         0          14m
heart-api-6f47679d58-dvhgw   0/1     Pending             0          0s
heart-api-6f47679d58-dvhgw   0/1     Pending             0          0s
heart-api-6f47679d58-x7rxt   1/1     Terminating         0          14m
heart-api-6f47679d58-dvhgw   0/1     ContainerCreating   0          0s
heart-api-6f47679d58-dvhgw   0/1     ContainerCreating   0          1s
heart-api-6f47679d58-dvhgw   0/1     Running             0          1s
heart-api-6f47679d58-dvhgw   0/1     Running             0          3s
heart-api-6f47679d58-x7rxt   0/1     Completed           0          14m
heart-api-6f47679d58-x7rxt   0/1     Completed           0          14m
heart-api-6f47679d58-x7rxt   0/1     Completed           0          14m
heart-api-6f47679d58-x7rxt   0/1     Completed           0          14m
heart-api-6f47679d58-dvhgw   0/1     Running             0          8s
heart-api-6f47679d58-dvhgw   1/1     Running             0          8s
```

Figure 15. *Self-healing: a deleted pod is replaced and Ready again in about 8 seconds while the other pod keeps serving.*

```
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease rollout restart deployment/heart-api
deployment.apps/heart-api restarted
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease rollout status deployment/heart-api
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
deployment "heart-api" successfully rolled out
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease rollout history deployment/heart-api
deployment.apps/heart-api
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>

(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease rollout undo deployment/heart-api
Warning: resource deployments/heart-api was previously managed with 'kubectl apply'. Rolling back will not update the kubectl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/heart-api rolled back
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease rollout status deployment/heart-api
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
deployment "heart-api" successfully rolled out
(.venv) PS C:\projects\heart-disease-mlops-assignment1\heart-disease-mlops> kubectl -n heart-disease rollout history deployment/heart-api
deployment.apps/heart-api
REVISION  CHANGE-CAUSE
1         <none>
3         <none>
4         <none>
```

Figure 16. *Zero-downtime rolling update (`rollout restart`) and rollback (`rollout undo`) with revision history.*

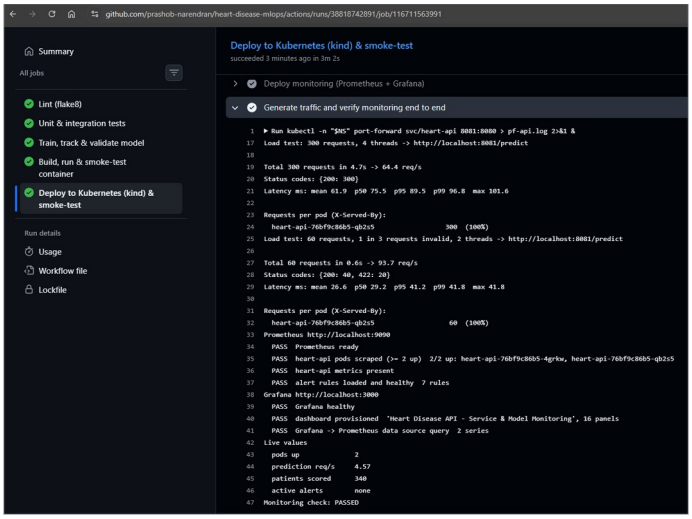


Figure 17. *CI Kubernetes job: traffic generated, Prometheus scrapes 2/2 pods, 7 rules loaded, dashboard provisioned, Monitoring check: PASSED.*

Appendix C. Virtual lab evidence

Further screenshots from the clean-room run on the BITS Prayogshala virtual lab (course MLOPS AIMLZG523, 17 October 2026), in addition to Figures 13 and 14. Every stage log is saved under `lab_proof/` on the lab machine; all 16 lab screenshots are in `screenshots/lab_*.png`.



Figure 18. *Lab VM: machine identity, dataset checksum, byte-identical fresh UCI download and 70 passed tests (top left); five deliberate failures stopped with non-zero exit codes (top right); hardened container healthy with 6/6 smoke checks and HTTP 422 for invalid input (bottom left); Swagger `/predict` with tracing headers and the MLflow UI with the lab runs and registered model v1 (bottom right).*